

Unidad 2

Diseño conceptual y lógico de la base de datos de Ecommify

Valentina Rodríguez Romero

Andrés Santiago Santafe Silva

Daniel Orlando Saavedra Fonnegra

Facultad de Ingeniería, Universidad de La Sabana

Diseño y Optimización de Bases de Datos

Miguel Alfonso Varela Fonseca

25 de mayo de 2026

1. Resumen ejecutivo

El presente proyecto desarrolla el diseño conceptual y lógico de una base de datos híbrida para Ecommify, una plataforma de comercio electrónico basada en el dataset Brazilian E-Commerce Public Dataset by Olist. La solución propuesta busca responder a las necesidades de procesamiento transaccional, gestión de catálogo flexible y análisis de grandes volúmenes de datos en un entorno escalable y de alta disponibilidad.

La arquitectura implementa un enfoque políglota utilizando PostgreSQL como motor relacional principal para el núcleo transaccional y MongoDB para el almacenamiento documental y analítico. PostgreSQL administra entidades críticas como clientes, pedidos, pagos, inventario y envíos, garantizando propiedades ACID, integridad referencial y control de concurrencia. Por su parte, MongoDB soporta el catálogo flexible de productos y las reseñas de usuarios, aprovechando estructuras semiestructuradas y patrones de modelado documental que facilitan la escalabilidad y optimización de lecturas masivas.

El modelo conceptual identifica entidades principales como Customers, Sellers, Products, Orders, Payments, Reviews, Shipments e Inventory, junto con sus relaciones y restricciones de negocio necesarias para asegurar consistencia e integridad operativa. El diseño relacional se encuentra normalizado hasta tercera forma normal (3FN), incorporando además capacidades avanzadas de PostgreSQL como JSONB, arrays, particionamiento por fechas y extensiones especializadas como pg_trgm y PostGIS para búsquedas tolerantes a errores y procesamiento geoespacial

2. Análisis de requisitos

2.1 Contexto de Ecommify

Ecommify es un sistema de comercio electrónico tipo marketplace, donde múltiples vendedores ofrecen productos a clientes finales. Los clientes pueden realizar pedidos, pagar con distintos métodos, recibir envíos y dejar reseñas sobre su experiencia de compra.

El sistema debe manejar información transaccional crítica como pedidos, pagos e inventario, pero también información flexible como especificaciones de productos, fotografías, comentarios y datos de búsqueda.

2.2 Identificación de usuarios

A partir del análisis realizado sobre el funcionamiento de plataformas de comercio electrónico y del dataset Brazilian E-commerce Public Dataset by Olist, se identificaron tres perfiles de usuario principales dentro del ecosistema de Ecommify.

El primero es el cliente, quien utiliza la plataforma para consultar productos, realizar compras, efectuar pagos y consultar el estado de sus pedidos. Este usuario requiere una experiencia rápida, segura y personalizada, además de acceso constante al catálogo de productos y al historial de compras. El segundo perfil corresponde al vendedor, responsable de publicar productos, gestionar inventario, atender pedidos y coordinar envíos. Este usuario necesita acceso a herramientas de administración comercial, monitoreo de ventas y actualización de stock en tiempo real. El tercer perfil identificado es el administrador de la plataforma, encargado de supervisar el funcionamiento general del sistema, validar operaciones críticas, monitorear métricas de rendimiento y garantizar la integridad de la información almacenada.

Cada perfil interactúa con diferentes módulos del sistema, lo que implica necesidades específicas de almacenamiento, consistencia y escalabilidad dentro de la arquitectura híbrida propuesta.

2.3 Necesidades de los usuarios

2.3.1 Cliente

La principal necesidad del cliente es poder consultar productos y realizar compras de forma rápida y segura desde cualquier dispositivo. Además, requiere acceso a información clara sobre precios, categorías, métodos de pago y estado de envío de los pedidos. El cliente también necesita realizar búsquedas eficientes incluso cuando existen errores tipográficos en el nombre de los productos, consultar su historial de compras y recibir una experiencia personalizada basada en disponibilidad y recomendaciones.

Adicionalmente, espera que los pagos y transacciones se procesen de manera confiable, evitando inconsistencias en pedidos o duplicidad de cobros.

2.3.2 Vendedor

La necesidad principal del vendedor es administrar eficientemente su catálogo de productos y controlar el inventario disponible. Requiere publicar productos con características variables dependiendo de la categoría, almacenar múltiples imágenes por producto y gestionar promociones temporales. También necesita monitorear pedidos recibidos, consultar métricas de ventas y optimizar los costos de envío según la ubicación geográfica de los clientes.

Debido a la naturaleza variable de los productos, el vendedor requiere flexibilidad en la estructura de almacenamiento de especificaciones y atributos.

2.3.3 Administrador

La principal necesidad del administrador es garantizar la estabilidad, disponibilidad y consistencia de la plataforma. Requiere monitorear transacciones, detectar inconsistencias, gestionar procesos de mantenimiento y analizar información histórica para la toma de decisiones.

También necesita acceso a métricas analíticas como ventas mensuales, segmentación de clientes y comportamiento de categorías, además de herramientas que permitan optimizar el rendimiento de consultas y almacenamiento. Por esta razón, el sistema debe soportar cargas híbridas OLTP y OLAP mediante estrategias avanzadas de PostgreSQL como particionamiento y vistas materializadas.

2.4 Requerimientos funcionales

2.4.1 RF01

Campo	Descripción
ID	RF01

Nombre	Registro de clientes
Tipo	Funcional
Prioridad	Alta
Actores	Cliente
Entrada	Nombre, correo, contraseña, dirección
Salida	Cuenta creada y validación exitosa
Relación	CLI01
Descripción	El sistema debe permitir el registro de clientes validando unicidad de correo electrónico y almacenamiento seguro de credenciales.

2.4.2 RF02

Campo	Descripción
ID	RF02
Nombre	Consulta de catálogo de productos
Tipo	Funcional
Prioridad	Alta

Actores	Cliente
Entrada	Categoría, búsqueda de producto
Salida	Lista de productos disponibles
Relación	CLI02
Descripción	El sistema debe permitir consultar productos mediante filtros y búsquedas tolerantes a errores tipográficos.

2.4.3 RF03

Campo	Descripción
ID	RF03
Nombre	Creación de pedidos
Tipo	Funcional
Prioridad	Alta
Actores	Cliente
Entrada	Productos seleccionados, dirección, método de pago
Salida	Pedido registrado y código de orden generado

Relación	CLI03
Descripción	El sistema debe permitir registrar pedidos con uno o múltiples productos asociados.

2.4.4 RF04

Campo	Descripción
ID	RF04
Nombre	Procesamiento de pagos
Tipo	Funcional
Prioridad	Alta
Actores	Cliente
Entrada	Método de pago, valor de transacción
Salida	Pago registrado y validado
Relación	CLI04
Descripción	El sistema debe registrar pagos asociados a pedidos garantizando consistencia transaccional.

2.4.5 RF05

Campo	Descripción
ID	RF05
Nombre	Gestión de inventario
Tipo	Funcional
Prioridad	Alta
Actores	Vendedor
Entrada	Cantidad disponible, producto
Salida	Inventario actualizado
Relación	VEN01
Descripción	El sistema debe permitir actualizar el stock de productos en tiempo real.

2.4.6 RF06

Campo	Descripción
ID	RF06
Nombre	Registro de productos
Tipo	Funcional

Prioridad	Alta
Actores	Vendedor
Entrada	Nombre, categoría, especificaciones, imágenes
Salida	Producto registrado
Relación	VEN02
Descripción	El sistema debe permitir registrar productos con atributos variables según su categoría.

2.4.7 RF07

Campo	Descripción
ID	RF07
Nombre	Gestión de envíos
Tipo	Funcional
Prioridad	Alta
Actores	Vendedor

Entrada	Dirección cliente, ubicación vendedor
Salida	Información de envío y costo estimado
Relación	VEN03
Descripción	El sistema debe calcular costos de envío utilizando información geográfica de clientes y vendedores.

2.4.8 RF08

Campo	Descripción
ID	RF08
Nombre	Registro de reseñas
Tipo	Funcional
Prioridad	Media
Actores	Cliente
Entrada	Calificación y comentario
Salida	Review registrada

Relación	CLI05
Descripción	El sistema debe permitir a los clientes registrar reseñas asociadas a pedidos completados.

2.5 Requerimientos no funcionales

2.5.1 RNF01

Campo	Descripción
ID	RNF01
Nombre	Consistencia transaccional
Tipo	No funcional
Prioridad	Alta
Actores	Cliente, vendedor
Entrada	Pedidos y pagos
Salida	Transacciones consistentes
Relación	RF03, RF04

Descripción	El sistema debe garantizar propiedades ACID para evitar pérdida o duplicación de transacciones.
-------------	---

2.5.2 RNF02

Campo	Descripción
ID	RNF02
Nombre	Tiempo de respuesta
Tipo	No funcional
Prioridad	Alta
Actores	Todos
Entrada	Consultas y transacciones
Salida	Respuesta menor a 3 segundos
Relación	RF02
Descripción	El sistema debe responder consultas frecuentes en menos de 3 segundos.

2.5.3 RNF03

Campo	Descripción
ID	RNF03
Nombre	Escalabilidad
Tipo	No funcional
Prioridad	Alta
Actores	Administrador
Entrada	Incremento de datos
Salida	Sistema estable bajo crecimiento
Relación	RF03, RF08
Descripción	El sistema debe soportar crecimiento continuo en pedidos, productos y reseñas.

2.5.4 RNF04

Campo	Descripción
ID	RNF04

Nombre	Disponibilidad
Tipo	No funcional
Prioridad	Alta
Actores	Todos
Entrada	Acceso a plataforma
Salida	Disponibilidad mínima del 99%
Relación	RF02
Descripción	La plataforma debe mantenerse disponible para consultas y compras en todo momento.

2.5.5 RNF05

Campo	Descripción
ID	RNF05
Nombre	Flexibilidad de datos
Tipo	No funcional

Prioridad	Alta
Actores	Vendedor
Entrada	Especificaciones variables
Salida	Productos almacenados correctamente
Relación	RF06
Descripción	El sistema debe soportar atributos dinámicos mediante JSONB y documentos NoSQL.

2.6 Análisis de requerimientos

El levantamiento de requerimientos funcionales y no funcionales permitió identificar que Ecommify debe responder a dos necesidades principales: garantizar la confiabilidad de las transacciones comerciales y ofrecer flexibilidad para manejar información variable del catálogo de productos.

La primera conclusión es que los procesos de creación de pedidos, procesamiento de pagos y gestión de inventario son los más críticos del sistema. Estos procesos requieren consistencia fuerte, ya que cualquier error podría generar cobros duplicados, pérdida de pedidos, inconsistencias en el stock o afectaciones económicas tanto para clientes como para vendedores. Por esta razón, estos módulos deben implementarse en PostgreSQL, aprovechando sus propiedades ACID, claves primarias, claves foráneas, restricciones de integridad y manejo transaccional.

La segunda conclusión es que el catálogo de productos requiere una estructura más flexible. En un marketplace, no todos los productos comparten los mismos atributos; por ejemplo, un celular puede tener memoria RAM, sistema operativo y capacidad de almacenamiento, mientras que un

mueble puede tener material, dimensiones y color. Por esto, el sistema debe permitir el uso de tipos avanzados como JSONB y arrays en PostgreSQL, además de considerar MongoDB para documentos de productos y reseñas.

La tercera conclusión es que Ecommify debe soportar tanto operaciones transaccionales como consultas analíticas. Los usuarios necesitan comprar y pagar rápidamente, mientras que los administradores requieren consultar métricas como ventas por mes, comportamiento por categoría y segmentación de clientes. Por ello, se propone una arquitectura híbrida con particionamiento de tablas, vistas materializadas y extensiones como PostGIS y pg_trgm.

2.7 Tabla de volúmenes de datos esperados

Entidad	Volumen estimado	Justificación
Customers	5.000.000	Usuarios registrados en la plataforma.
Sellers	500.000	Vendedores activos y potenciales.
Products	2.000.000	Catálogo amplio con diferentes categorías.
Orders	10.000.000	Pedidos históricos y activos.
Order Items	30.000.000	Un pedido puede contener varios productos.
Payments	12.000.000	Un pedido puede tener uno o más pagos.
Reviews	15.000.000	Reseñas generadas por clientes.

Inventory	5.000.000	Stock por producto y vendedor.
Geolocations	1.000.000+	Ubicaciones de clientes, vendedores y códigos postales.
Shipments	10.000.000	Información de envío por pedido.

3. Diseño conceptual

3.1 Descripción general del modelo conceptual

El modelo conceptual de Ecommify representa las entidades principales de un sistema de comercio electrónico tipo marketplace. En este modelo, los clientes realizan pedidos, los pedidos contienen uno o varios productos, los productos son ofrecidos por vendedores y cada pedido puede tener pagos, envíos y reseñas asociadas.

El diseño busca mantener una estructura normalizada, evitando redundancias innecesarias y separando correctamente las responsabilidades de cada entidad. Las entidades transaccionales principales se modelan en PostgreSQL, mientras que la información más flexible, como especificaciones variables de productos y comentarios de reseñas, puede complementarse con MongoDB.

3.2 Entidades principales del dominio

Entidad	Descripción
Customer	Representa a los clientes registrados en la plataforma.

Seller	Representa a los vendedores que ofrecen productos.
Product	Representa los productos disponibles en el marketplace.
Category	Clasifica los productos según su tipo.
Order	Representa una compra realizada por un cliente.
OrderItem	Detalla los productos incluidos dentro de cada pedido.
Payment	Almacena los pagos asociados a un pedido.
Shipment	Contiene información relacionada con el envío del pedido.
Review	Representa la calificación y opinión del cliente sobre una compra.
Inventory	Controla la disponibilidad de productos por vendedor.
Geolocation	Almacena datos geográficos de clientes y vendedores.

3.3 Relaciones entre entidades

Relación	Cardinalidad	Descripción
----------	--------------	-------------

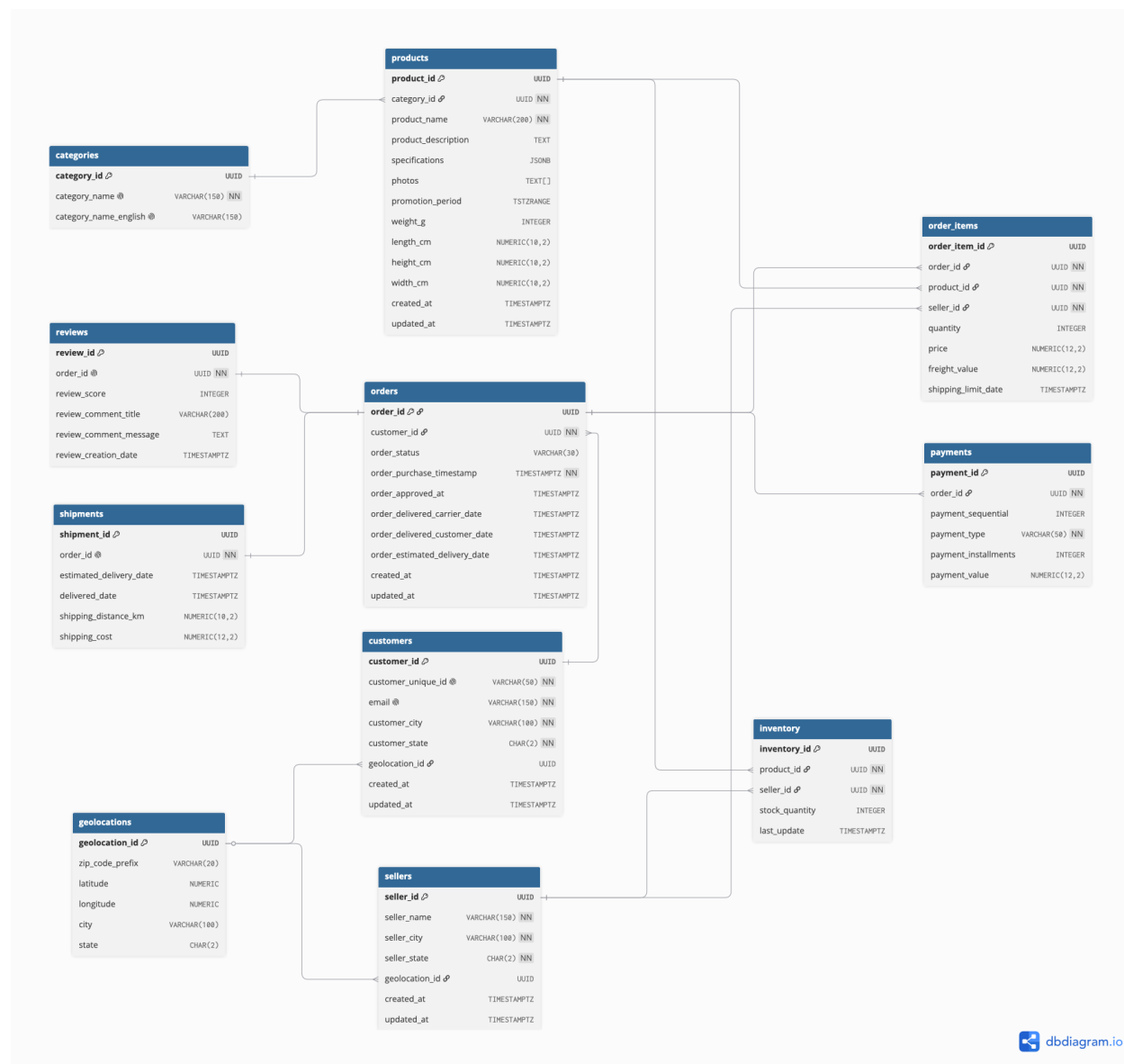
Customer - Order	1:N	Un cliente puede realizar muchos pedidos, pero cada pedido pertenece a un solo cliente.
Order - OrderItem	1:N	Un pedido puede contener varios productos.
Product - OrderItem	1:N	Un producto puede aparecer en muchos pedidos.
Seller - Product	1:N	Un vendedor puede publicar varios productos.
Category - Product	1:N	Una categoría puede agrupar muchos productos.
Order - Payment	1:N	Un pedido puede tener uno o varios pagos.
Order - Shipment	1:1	Cada pedido tiene información de envío asociada.
Order - Review	1:1	Cada pedido puede tener una reseña.
Seller - Inventory	1:N	Un vendedor puede gestionar inventario de varios productos.

Product - Inventory	1:N	Un producto puede tener inventario asociado por vendedor.
---------------------	-----	---

3.4 Restricciones de negocio identificadas

- Un cliente debe tener un correo electrónico único.
- Un pedido siempre debe estar asociado a un cliente.
- Un pedido debe contener al menos un producto.
- Un pago debe estar asociado a un pedido existente.
- El valor de un pago no puede ser negativo.
- La cantidad de productos en inventario no puede ser negativa.
- Una reseña solo puede registrarse para un pedido existente.
- Un producto debe pertenecer a una categoría.
- Un vendedor puede ofrecer múltiples productos.
- El estado de un pedido debe pertenecer a un conjunto controlado de valores.
- La fecha de entrega no puede ser anterior a la fecha de compra.
- Los datos geográficos deben permitir calcular distancias entre clientes y vendedores.

3.5 Diagrama ER sugerido



3.6 Atributos principales por entidad

3.6.1 Customer

Atributo	Descripción
----------	-------------

customer_id	Identificador único del cliente.
customer_unique_id	Identificador único alternativo del cliente.
customer_city	Ciudad del cliente.
customer_state	Estado o región del cliente.
geolocation_id	Referencia a la ubicación geográfica.

3.6.2 Seller

Atributo	Descripción
seller_id	Identificador único del vendedor.
seller_city	Ciudad del vendedor.
seller_state	Estado o región del vendedor.
geolocation_id	Referencia a la ubicación geográfica.

3.6.3 Product

Atributo	Descripción
----------	-------------

product_id	Identificador único del producto.
category_id	Categoría del producto.
product_name	Nombre del producto.
specifications	Especificaciones variables del producto.
photos	Lista de imágenes del producto.

3.6.4 Order

Atributo	Descripción
order_id	Identificador único del pedido.
customer_id	Cliente que realiza el pedido.
order_status	Estado del pedido.
order_purchase_timestamp	Fecha de compra.
order_delivered_customer_date	Fecha de entrega al cliente.
created_at	Fecha de creación del registro.
updated_at	Fecha de última actualización.

3.6.5 OrderItem

Atributo	Descripción
order_item_id	Identificador del ítem dentro del pedido.
order_id	Pedido asociado.
product_id	Producto comprado.
seller_id	Vendedor asociado.
quantity	Cantidad comprada.
price	Precio del producto.
freight_value	Valor del envío.

3.6.6 Payment

Atributo	Descripción
payment_id	Identificador único del pago.
order_id	Pedido asociado.

payment_type	Tipo de pago.
payment_installments	Número de cuotas.
payment_value	Valor pagado.

3.6.7 Review

Atributo	Descripción
review_id	Identificador de la reseña.
order_id	Pedido evaluado.
review_score	Calificación del cliente.
review_comment_title	Título del comentario.
review_comment_message	Mensaje de la reseña.
review_creation_date	Fecha de creación.

3.6.8 Shipment

Atributo	Descripción
shipment_id	Identificador del envío.

order_id	Pedido enviado.
estimated_delivery_date	Fecha estimada de entrega.
delivered_date	Fecha real de entrega.
shipping_distance_km	Distancia aproximada de envío.
shipping_cost	Costo estimado del envío.

3.6.9 Inventory

Atributo	Descripción
inventory_id	Identificador del inventario.
product_id	Producto asociado.
seller_id	Vendedor asociado.
stock_quantity	Cantidad disponible.
last_update	Última actualización del inventario.

3.6.10 Geolocation

Atributo	Descripción
geolocation_id	Identificador geográfico.
zip_code_prefix	Prefijo del código postal.
latitude	Latitud.
longitude	Longitud.
city	Ciudad.
state	Estado o región.

4. Diseño lógico - Módulo PostgreSQL

4.1 Descripción general del diseño lógico

El módulo PostgreSQL de Ecommify almacena la información transaccional crítica de la plataforma. Este módulo incluye clientes, vendedores, pedidos, ítems de pedido, pagos, envíos, inventario y geolocalización.

La decisión de utilizar PostgreSQL se fundamenta en la necesidad de garantizar consistencia, integridad referencial y control transaccional en procesos sensibles como la creación de pedidos, el procesamiento de pagos y la actualización de inventario. Estos procesos requieren propiedades ACID, claves primarias, claves foráneas, restricciones de negocio y validaciones a nivel de base

de datos. El esquema propuesto se encuentra normalizado mínimo hasta tercera forma normal, ya que cada tabla representa una única entidad del dominio, los atributos dependen directamente de su clave primaria y no se almacenan dependencias transitivas innecesarias. Además, se integran capacidades avanzadas de PostgreSQL como JSONB, arrays, tipos compuestos, rangos temporales, particionamiento y extensiones especializadas.

4.2 Esquema relacional normalizado

El diseño relacional se estructura en las siguientes tablas principales:

Tabla	Propósito
customers	Almacenar información de clientes.
sellers	Almacenar información de vendedores.
categories	Clasificar productos.
products	Registrar productos y atributos principales.
orders	Registrar pedidos realizados por clientes.
order_items	Detallar los productos incluidos en cada pedido.
payments	Registrar pagos asociados a pedidos.
shipments	Registrar información de envío.
inventory	Controlar existencias por producto y vendedor.

reviews	Registrar reseñas asociadas a pedidos.
geolocations	Almacenar datos geográficos.

4.3 Tablas, columnas, tipos de datos y constraints

4.3.1 Tabla customers

Columna	Tipo de dato	Clave / Constraint	Descripción
customer_id	UUID	PK	Identificador único del cliente.
customer_unique_id	VARCHAR(50)	UNIQUE, NOT NULL	Identificador único alternativo.
email	VARCHAR(150)	UNIQUE, NOT NULL	Correo del cliente.
customer_city	VARCHAR(100)	NOT NULL	Ciudad del cliente.
customer_state	CHAR(2)	NOT NULL	Estado o región.

geolocation_id	UUID	FK	Referencia geográfica.
created_at	TIMESTAMPTZ	DEFAULT now()	Fecha de creación.
updated_at	TIMESTAMPTZ	DEFAULT now()	Fecha de actualización.

4.3.2 Tabla sellers

Columna	Tipo de dato	Clave / Constraint	Descripción
seller_id	UUID	PK	Identificador único del vendedor.
seller_name	VARCHAR(150)	NOT NULL	Nombre comercial del vendedor.
seller_city	VARCHAR(100)	NOT NULL	Ciudad del vendedor.
seller_state	CHAR(2)	NOT NULL	Estado o región.
geolocation_id	UUID	FK	Referencia geográfica.
created_at	TIMESTAMPTZ	DEFAULT now()	Fecha de creación.

updated_at	TIMESTAMPTZ	DEFAULT now()	Fecha de actualización.
------------	-------------	------------------	-------------------------

4.3.3 Tabla categories

Columna	Tipo de dato	Clave / Constraint	Descripción
category_id	UUID	PK	Identificador único de categoría.
category_name	VARCHAR(150)	UNIQUE, NOT NULL	Nombre original de la categoría.
category_name_english	VARCHAR(150)	UNIQUE	Nombre traducido de la categoría.

4.3.4 Tabla products

Columna	Tipo de dato	Clave / Constraint	Descripción
product_id	UUID	PK	Identificador único del producto.

category_id	UUID	FK, NOT NULL	Categoría del producto.
product_name	VARCHAR(200)	NOT NULL	Nombre del producto.
product_description	TEXT	NULL	Descripción del producto.
specifications	JSONB	DEFAULT '{'	Atributos variables del producto.
photos	TEXT[]	NULL	URLs o nombres de imágenes.
promotion_period	TSTZRANGE	NULL	Periodo de promoción.
weight_g	INTEGER	CHECK weight_g >= 0	Peso en gramos.
length_cm	NUMERIC(10,2)	CHECK length_cm >= 0	Largo.
height_cm	NUMERIC(10,2)	CHECK height_cm >= 0	Alto.

width_cm	NUMERIC(10,2)	CHECK width_cm >= 0	Ancho.
created_at	TIMESTAMPTZ	DEFAULT now()	Fecha de creación.
updated_at	TIMESTAMPTZ	DEFAULT now()	Fecha de actualización.

4.3.5 Tabla orders

Columna	Tipo de dato	Clave / Constraint	Descripción
order_id	UUID	PK	Identificador del pedido.
customer_id	UUID	FK, NOT NULL	Cliente que realiza el pedido.
order_status	VARCHAR(30)	CHECK	Estado del pedido.
order_purchase_timestamp	TIMESTAMPTZ	NOT NULL	Fecha de compra.

order_approved_at	TIMESTAMPTZ	NULL	Fecha de aprobación.
order_delivered_carrier_date	TIMESTAMPTZ	NULL	Fecha de entrega al transportador.
order_delivered_customer_date	TIMESTAMPTZ	NULL	Fecha de entrega al cliente.
order_estimated_delivery_date	TIMESTAMPTZ	NULL	Fecha estimada de entrega.
created_at	TIMESTAMPTZ	DEFAULT now()	Fecha de creación.
updated_at	TIMESTAMPTZ	DEFAULT now()	Fecha de actualización

4.3.6 Tabla *order_items*

Columna	Tipo de dato	Clave / Constraint	Descripción
order_item_id	UUID	PK	Identificador del ítem.
order_id	UUID	FK, NOT NULL	Pedido asociado.

product_id	UUID	FK, NOT NULL	Producto comprado.
seller_id	UUID	FK, NOT NULL	Vendedor del producto.
quantity	INTEGER	CHECK quantity > 0	Cantidad comprada.
price	NUMERIC(12,2)	CHECK price >= 0	Precio del producto.
freight_value	NUMERIC(12,2)	CHECK freight_value >= 0	Costo de envío.
shipping_limit_date	TIMESTAMP TZ	NULL	Fecha límite de envío.

4.3.7 Tabla payments

Columna, Tipo de dato, Clave / Constraint, Descripción

payment_id, UUID, PK, Identificador único del pago.

order_id, UUID, FK, NOT NULL, Pedido asociado.

payment_sequential, INTEGER, CHECK payment_sequential > 0, Secuencia para acomodar múltiples pagos en un mismo pedido.

payment_type, VARCHAR(50), NOT NULL, Método de pago elegido por el cliente (ej. credit_card, boleto).

payment_installments, INTEGER, CHECK payment_installments >= 0, Número de cuotas elegidas por el cliente.

payment_value, NUMERIC(12,2), CHECK payment_value >= 0, Valor de la transacción.

4.3.8 Tabla shipments

Columna, Tipo de dato, Clave / Constraint, Descripción

shipment_id, UUID, PK, Identificador único del envío.

order_id, UUID, FK, UNIQUE, NOT NULL, Pedido asociado (Garantiza relación 1:1 con orders).

estimated_delivery_date, TIMESTAMPTZ, NOT NULL, Fecha estimada de entrega calculada por el sistema.

delivered_date, TIMESTAMPTZ, NULL, Fecha real en la que el cliente recibió el producto.

shipping_distance_km, NUMERIC(10,2), CHECK shipping_distance_km >= 0, Distancia aproximada calculada entre vendedor y cliente.

shipping_cost, NUMERIC(12,2), CHECK shipping_cost >= 0, Costo final estimado del envío cobrado por logística.

4.3.9 Tabla inventory

Columna, Tipo de dato, Clave / Constraint, Descripción

inventory_id, UUID, PK, Identificador único del registro de inventario.

product_id, UUID, FK, NOT NULL, Producto asociado.

seller_id, UUID, FK, NOT NULL, Vendedor dueño del inventario.

stock_quantity, **INTEGER**, **CHECK** *stock_quantity* >= 0, Cantidad disponible del producto en bodega (No puede ser negativa).

last_update, **TIMESTAMPTZ**, **DEFAULT** now(), Última fecha y hora de actualización del stock.

4.3.10 Tabla reviews

Columna, Tipo de dato, Clave / Constraint, Descripción

review_id, **UUID**, **PK**, Identificador único de la reseña.

order_id, **UUID**, **FK**, **UNIQUE**, **NOT NULL**, Pedido evaluado (Asegura relación de integridad 1:1 por pedido).

review_score, **INTEGER**, **CHECK** (*review_score* >= 1 AND *review_score* <= 5), Calificación del cliente en la escala estándar de 1 a 5.

review_comment_title, **VARCHAR**(200), **NULL**, Título o encabezado corto del comentario.

review_comment_message, **TEXT**, **NULL**, Mensaje detallado u opinión de la experiencia de compra.

review_creation_date, **TIMESTAMPTZ**, **DEFAULT** now(), Fecha de creación del registro de la reseña.

4.3.11 Tabla geolocations

Columna, Tipo de dato, Clave / Constraint, Descripción

geolocation_id, **UUID**, **PK**, Identificador geográfico único para indexación interna.

zip_code_prefix, **VARCHAR**(20), **NOT NULL**, Prefijo del código postal (Estructura base del dataset Olist).

latitude, **NUMERIC**(10,8), **NOT NULL**, Coordenada geográfica de latitud.

longitude, NUMERIC(11,8), NOT NULL, Coordenada geográfica de longitud.

city, VARCHAR(100), NOT NULL, Nombre de la ciudad o localidad.

state, CHAR(2), NOT NULL, Iniciales/código del estado o región administrativa.

4.4 Justificación de Extensiones a Utilizar en PostgreSQL

Para soportar las necesidades de búsqueda avanzada y procesamiento geoespacial exigidas por las reglas de negocio de Ecommify, se habilitarán dos extensiones nativas en el clúster de PostgreSQL: pg_trgm y PostGIS.

4.4.1 Extensión pg_trgm (Trigram Matching)

Propósito técnico: Descomponer cadenas de texto en secuencias de tres caracteres consecutivos (trigramas) para medir la similitud entre palabras mediante el índice de Jaccard.

Justificación en Ecommify: El Requerimiento Funcional RF02 (Consulta de catálogo de productos) exige explícitamente que el sistema realice búsquedas tolerantes a errores tipográficos. Con un volumen proyectado de 2,000,000 de productos, realizar consultas tradicionales con LIKE o ILIKE provocaría escaneos secuenciales completos (Sequential Scans), colapsando el rendimiento de la base de datos.

Implementación: Al activar pg_trgm, se indexarán las columnas product_name y product_description mediante índices de tipo GIN (Generalized Inverted Index) o GiST. Esto permitirá resolver consultas con errores ortográficos (por ejemplo, buscar "smarfone" y encontrar "smartphone") en milisegundos, garantizando el cumplimiento del RNF02 (tiempo de respuesta menor a 3 segundos).

4.4.2 Extensión PostGIS

Propósito técnico: Añadir soporte para objetos geográficos (puntos, líneas, polígonos) permitiendo ejecutar consultas SQL espaciales y cálculos de distancias basados en geometría elipsoidal exacta.

Justificación en Ecommify: La restricción de negocio de la sección 3.4 y el requerimiento RF07 (Gestión de envíos) estipulan que el sistema debe calcular costos de envío utilizando la información geográfica de clientes y vendedores.

Implementación: En lugar de tratar la latitud y longitud de la tabla geolocations como simples números flotantes (que requerirían complejas fórmulas trigonométricas de Haversine en el código de la aplicación), PostGIS permite unificar estas coordenadas en un tipo de dato nativo GEOMETRY(Point, 4326).

Gracias a esto, el cálculo de la distancia entre un cliente y un vendedor se reduce a una función nativa optimizada:

SELECT ST_DistanceSphere(vendedor.geom, cliente.geom) / 1000 AS distancia_km;

Esto se complementa con índices espaciales GiST para procesar eficientemente más de 1,000,000 de coordenadas geográficas.

4.5 Estrategia de Particionamiento de Datos

Con un volumen estimado de 10,000,000 de pedidos (Orders) históricos y activos, la tabla orders se convertirá rápidamente en un cuello de botella si se mantiene de forma monolítica. Para mitigar esto, se aplicará una estrategia de Particionamiento por Rango (Range Partitioning) basado en Fechas, tomando como llave de partición la columna order_purchase_timestamp.

4.5.1 ¿Por qué particionar específicamente por fechas?

Patrón de Acceso a los Datos (Balance OLTP vs. OLAP): En un marketplace, el 90% de las operaciones del día a día (transaccional/OLTP) acceden a datos recientes: clientes consultando

pedidos del último mes, vendedores procesando despachos de la semana o pasarelas validando pagos actuales. Por otro lado, las órdenes de años anteriores tienen un comportamiento estrictamente de lectura analítica (OLAP) por parte del administrador. Al particionar por fechas (por ejemplo, particiones mensuales o trimestrales), las consultas operacionales solo escanearán los bloques de datos del periodo vigente.

Poda de Particiones (Partition Pruning): PostgreSQL cuenta con un optimizador inteligente.

Si un usuario consulta su historial de compras del último mes, el motor de la base de datos ignora por completo las particiones de los meses o años pasados. No consume memoria ram ni disco analizando millones de registros viejos. Esto asegura que el rendimiento de las consultas frecuentes de los clientes se mantenga constante a lo largo de los años, cumpliendo con la alta disponibilidad (RNF04) y tiempos de respuesta (RNF02).

Ciclo de Vida del Dato y Mantenimiento Eficiente: Hacer mantenimiento sobre una tabla de 10 millones de filas es crítico. Si el administrador decide archivar las órdenes que tienen más de 5 años de antigüedad, ejecutar un comando tradicional como `DELETE FROM orders WHERE fecha < X` bloquearía la tabla por horas, generaría un uso masivo del log de transacciones (WAL) y provocaría fragmentación (bloat). Con particionamiento por fechas, el proceso es inmediato y no bloquea el sistema, ya que se reduce a una operación de metadatos:

ALTER TABLE orders DETACH PARTITION orders_2021_m05;

DROP TABLE orders_2021_m05;

4.5.2 Diseño Preliminar de la Implementación SQL

La tabla principal se declarará como contenedora lógica de los rangos espaciales de tiempo:

-- Tabla Padre declarada para particionamiento CREATE TABLE

orders (order_id UUID NOT NULL, customer_id UUID NOT NULL, order_status VARCHAR(30), order_purchase_timestamp TIMESTAMPTZ NOT NULL, PRIMARY KEY (order_id, order_purchase_timestamp) -- La llave de partición debe ser parte de la PK) PARTITION BY RANGE (order_purchase_timestamp);

-- Ejemplo de creación de una partición mensual para Mayo de 2026 CREATE TABLE

orders_2026_m05 PARTITION OF orders FOR VALUES FROM ('2026-05-01 00:00:00+00') TO ('2026-06-01 00:00:00+00');

5. Diseño Lógico Preliminar - Módulo MongoDB

5.1 Justificación Técnica Comparativa (PostgreSQL vs. MongoDB)

La arquitectura híbrida de Ecommify surge de la necesidad de balancear una consistencia transaccional absoluta con una alta flexibilidad en el almacenamiento de datos dinámicos. La distribución de responsabilidades entre ambos motores se define bajo los siguientes criterios técnicos:

PostgreSQL (Módulo Transaccional Core - ACID): Se encarga de las entidades cuyo estado crítico requiere restricciones de integridad referencial fuerte, manejo de concurrencia y prevención de anomalías monetarias o de stock. Los procesos de creación de pedidos, procesamiento de pagos y gestión de inventario permanecen estrictamente en el modelo relacional para garantizar que no ocurran cobros duplicados ni sobreventas.

MongoDB (Módulo de Catálogo y Feedback - NoSQL): Se utiliza para mitigar las limitaciones de los esquemas rígidos frente a datos altamente polimórficos o semiestructurados. En lugar de recurrir a antipatrones relacionales complejos (como Entity-Attribute-Value o tablas con cientos

de columnas vacías), MongoDB almacena de manera nativa los atributos variables de las distintas categorías de productos y los extensos volúmenes de texto de las reseñas de usuarios.

5.2 Colecciones Principales Identificadas

Para cubrir el espectro flexible del marketplace se han estructurado dos colecciones principales dentro del clúster de MongoDB:

`products_catalog`: Almacena la información detallada de los productos disponibles. Permite la coexistencia de múltiples esquemas conceptuales bajo una misma colección (por ejemplo, un artículo de tecnología y una prenda de vestir).

`order_reviews`: Repositorio optimizado para lecturas masivas de las calificaciones y opiniones de los compradores. Al estar desacoplado del motor relacional principal, evita que la alta carga analítica o de lectura de comentarios afecte el rendimiento de la tabla transaccional de órdenes de compra.

5.3 Esquema de Documentos (Ejemplos en JSON)

A continuación, se presentan los modelos de documentos propuestos en formato JSON, reflejando cómo se manejan los tipos compuestos y la anidación nativa:

5.3.1 Ejemplo de Documento en la Colección `products_catalog`

```
{
  "_id": {"$oid": "665239a1f1b2c3d4e5f6a7b8"},
  "product_id": "d9b2b1a0-c3d4-4e5f-a6b7-c8d9e0f1a2b3",
  "product_name": "Smartphone X Pro Max",
  "category": {
```

```

"category_id": "e0a1b2c3-d4e5-4f6a-7b8c-9d0e1f2a3b4",
"name": "Celulares y Smartphones",
"name_english": "Cellphones & Smartphones"
},
"specifications": [
  {"attribute": "Memoria RAM", "value": "12 GB"},
  {"attribute": "Almacenamiento", "value": "256 GB"},
  {"attribute": "Procesador", "value": "Octa-Core 3.4 GHz"},
  {"attribute": "Batería", "value": "5000 mAh"}
],
"photos": [
  "https://cdn.ecommify.com/products/tech/img_front.jpg",
  "https://cdn.ecommify.com/products/tech/img_back.jpg"
],
"dimensions": {
  "weight_g": 221,
  "length_cm": 16.07,
  "height_cm": 0.83,
  "width_cm": 7.76
},
"created_at": "2026-05-25T14:30:00Z"
}

```

5.3.2 Ejemplo de Documento en la Colección order_reviews

```
{
  "_id": {"$oid": "665239b5f1b2c3d4e5f6a7b9"},
  "review_id": "f5e4d3c2-b1a0-4f5e-a6b7-c8d9e0f1a2b3",
  "order_id": "fa65dad1-b0e8-18e3-ccc5-cb0e39231352",
  "customer_summary": {
    "customer_id": "1a2b3c4d-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
    "username": "val_rodriguez"
  },
  "review_score": 5,
  "review_comment_title": "Excelente rendimiento y entrega puntual",
  "review_comment_message": "El dispositivo superó mis expectativas. La pantalla es increíble
y la batería dura todo el día sin problemas. Llegó un día antes de lo estimado.",
  "review_creation_date": "2026-05-25T20:15:00Z"
}
```

5.4 Patrones de Modelado a Aplicar

Para maximizar la eficiencia en las operaciones de lectura y escritura en el módulo NoSQL, se implementarán de forma estricta los siguientes patrones de modelado de datos en MongoDB:

5.4.1 Patrón de Atributo (Attribute Pattern)

Aplicación: Se utiliza dentro de la colección `products_catalog` en el arreglo `specifications`.

Justificación: Dado que el inventario del marketplace cuenta con atributos impredecibles y variables por categoría (por ejemplo, ropa requiere "Talla" y "Material", mientras que tecnología requiere "RAM" y "Voltaje"), este patrón unifica la estructura interna en pares clave-valor (attribute / value). Esto permite crear índices compuestos sobre campos variables de forma

genérica, optimizando las consultas de filtrado sin necesidad de alterar el esquema global cada vez que ingresa una nueva categoría al catálogo.

5.4.2 Patrón de Referencia Extendida (Extended Reference Pattern)

Aplicación: Se utiliza en la colección `order_reviews` al incluir el objeto embebido `customer_summary` y en `products_catalog` con los datos clave de la categoría.

Justificación: Las operaciones de lectura más frecuentes en una plataforma de e-commerce implican desplegar las reseñas junto al nombre de usuario que la escribió, o los productos junto a su categoría principal. En lugar de forzar a la aplicación a hacer un cruce de datos (lookup o join) hacia PostgreSQL en cada renderizado de página, se duplican intencionalmente los datos estáticos de mayor prioridad (como `username` o `name` de la categoría). Al tratarse de información que cambia con una frecuencia casi nula, el costo de mitigar la desnormalización es mínimo comparado con la ganancia masiva en velocidad de lectura de la interfaz de usuario.

6. Decisiones Arquitectónicas Justificadas

6.1 Matriz de Decisión por Entidad (Arquitectura Híbrida)

Ecommify implementa una Arquitectura Híbrida/Polígota (PostgreSQL + MongoDB) para responder de forma óptima a las necesidades de consistencia transaccional y flexibilidad del catálogo.

A continuación, se detalla la distribución de las entidades principales según su naturaleza operativa:

Entidad, Motor Asignado, Criterio de Selección / Justificación Técnica

Customers, Sellers, Geolocations, PostgreSQL, Requieren integridad referencial estricta y llaves foráneas para el correcto funcionamiento de operaciones relacionales y cálculo de distancias con PostGIS.

Orders, Order Items, Payments, PostgreSQL, Exigen cumplimiento estricto de propiedades ACID. Previene anomalías críticas como duplicidad de cobros o inconsistencias en pedidos con alto volumen (10,000,000 de órdenes).

Inventory, PostgreSQL, Requiere consistencia fuerte inmediata y bloqueos transaccionales para evitar la sobreventa (overselling) de productos en tiempo real.

Products (Catálogo), MongoDB, Manejo de datos altamente polimórficos. Permite almacenar atributos y especificaciones sumamente variables por categoría sin alterar un esquema rígido.

Reviews, MongoDB, Aisladas del motor relacional principal para soportar un crecimiento masivo (15,000,000 de registros) sin degradar el rendimiento de las consultas operacionales transaccionales.

6.2 Análisis del Teorema CAP Aplicado

Ante la presencia de una partición de red (Partition Tolerance), el sistema híbrido de Ecommify divide su comportamiento de la siguiente manera:

- Módulo Transaccional Core (PostgreSQL) → Enfoque CP (Consistencia y Tolerancia a Particiones):** En los procesos de cobros, creación de pedidos e inventario, la prioridad absoluta es la consistencia de los datos. Si ocurre una desconexión en el clúster, el sistema prefiere rechazar temporalmente la transacción antes que procesar un estado inconsistente (como vender un producto sin stock).
- Módulo de Catálogo y Feedback (MongoDB) → Enfoque AP (Disponibilidad y Tolerancia a Particiones):** Para la navegación de productos y consulta de reseñas, el sistema prioriza el requerimiento RNF04 (disponibilidad del 99%). Si un nodo NoSQL se aísla, se prefiere servir datos ligeramente desactualizados (Consistencia Eventual) antes que impedir que el cliente visualice el catálogo o sus recomendaciones.

6.3 Flujo de Sincronización mediante CDC (Change Data Capture)

Para comunicar ambos motores sin generar un acoplamiento directo en el código de la aplicación, se implementa una estrategia de Captura de Datos en Cambio (CDC) utilizando Debezium y un bus de eventos con Apache Kafka.

[PostgreSQL (WAL)] → [Debezium Connector] → [Apache Kafka Topic] → [Consumer Application] → [MongoDB]

1. **Captura:** Cada vez que un proceso transaccional en PostgreSQL inserta o modifica datos de un producto o inventario, el cambio se escribe en el log de transacciones nativo (WAL - Write-Ahead Logging).
2. **Streaming:** Debezium lee el WAL de forma asíncrona en tiempo real (milisegundos) y publica un evento estructurado en un tópico dedicado de Apache Kafka.
3. **Consumo e Impacto:** Una aplicación consumidora ligera extrae el evento de Kafka, transforma los datos al formato JSON requerido y actualiza la colección correspondiente en MongoDB.

6.4 Trade-offs y Estrategias de Mitigación

Desventaja (Trade-off): Alta Complejidad Operativa y Costo de Infraestructura. Administrar de forma simultánea un clúster relacional, una base de datos documental y una plataforma de streaming de eventos (Kafka) eleva la curva de aprendizaje y los costos de mantenimiento de servidores.

Estrategia de Mitigación: Para contrarrestar esta complejidad y garantizar la estabilidad exigida por el administrador, se optará por el uso de servicios gestionados en la nube (como AWS RDS para PostgreSQL, MongoDB Atlas y Confluent Cloud para Kafka). Esto reduce la carga

administrativa de parches, copias de seguridad y escalamiento vertical/horizontal, permitiendo al equipo enfocarse exclusivamente en la optimización del modelo de datos y consultas.

6.5 Registro de Decisiones Arquitectónicas (ADRs)

ADR 01: Adopción de una Arquitectura de Base de Datos Híbrida (PostgreSQL + MongoDB)

Contexto

El sistema Ecommify debe procesar operaciones transaccionales críticas como la creación de pedidos (10,000,000 estimados), cobro de pagos e inventarios. Al mismo tiempo, debe gestionar datos altamente polimórficos como las especificaciones de productos y un volumen masivo de opiniones de usuarios (15,000,000 de reseñas estimadas). Un esquema puramente relacional degrada el rendimiento al lidiar con atributos dinámicos por categoría , mientras que un modelo NoSQL exclusivo compromete la consistencia ACID requerida en el flujo financiero y de stock.

Decisión

Se implementará una arquitectura de persistencia polígloa/híbrida. Se delega a PostgreSQL el almacenamiento del núcleo operacional core (clientes, vendedores, pedidos, pagos, envíos e inventario) para asegurar integridad referencial y transaccional fuerte. En paralelo, se adopta MongoDB para el almacenamiento documental del catálogo flexible de productos y el repositorio analítico de reseñas.

Consecuencias

Positivas: Consistencia fuerte inmediata en procesos monetarios y de inventario, evitando cobros duplicados o pérdidas de pedidos.

Positivas: Flexibilidad total en el catálogo para registrar productos con estructuras de datos variables sin alterar esquemas rígidos.

Positivas: Aislamiento de la carga masiva de lectura de reseñas para proteger el rendimiento transaccional principal.

Negativas: Incremento en la complejidad del desarrollo al tener que administrar dos paradigmas de almacenamiento distintos de forma simultánea.

ADR 02: Estrategia de Sincronización de Datos mediante CDC (Change Data Capture)

Contexto

Derivado de la adopción de la arquitectura híbrida (ADR 01), los datos que se registran y validan en el núcleo relacional de PostgreSQL (como nuevos productos o actualizaciones de estado) deben verse reflejados en las colecciones de MongoDB para mantener la consistencia del catálogo de cara al usuario final. El uso de escrituras duales sincrónicas desde la aplicación introduce un fuerte acoplamiento y riesgos de inconsistencia ante fallos de red, mientras que los procesos por lotes (Batch ETL) rompen la necesidad de sincronización en tiempo real exigida por el negocio.

Decisión

Implementar un flujo de sincronización asíncrono basado en Captura de Datos en Cambio (CDC) de extremo a extremo. Se utilizará Debezium como conector para leer directamente el log de transacciones nativo (Write-Ahead Logging - WAL) de PostgreSQL ante cualquier evento de cambio, publicando dichos estados en tópicos de Apache Kafka. Una aplicación consumidora ligera procesará estos mensajes y actualizará las colecciones de MongoDB de manera asíncrona.

Consecuencias

Positivas: Sincronización casi instantánea (milisegundos), garantizando que las actualizaciones de catálogo e inventario se reflejen en tiempo real para clientes y vendedores.

Positivas: Desacoplamiento absoluto a nivel de aplicación; el flujo principal de compras (OLTP) no se ve afectado por la latencia de red del segundo motor.

Positivas: Resiliencia y tolerancia a fallos. Si el clúster de MongoDB experimenta una caída temporal, los eventos permanecen persistidos de forma segura en la cola de Kafka y se procesan automáticamente al restablecerse el servicio.

Negativas: Mayor costo de infraestructura y complejidad operativa debido al aprovisionamiento, configuración y monitoreo del clúster de mensajería (Kafka) y los conectores de Debezium.

7. Anexos

Para garantizar la legibilidad de este informe técnico y facilitar el despliegue directo de la arquitectura, todos los artefactos de código, el modelado extendido y los scripts de automatización se encuentran alojados y versionados en el repositorio oficial de control de versiones del proyecto.

7.1 Enlace al Repositorio Oficial

- **Proyecto:** Ecommify Database Design
- **Link de Acceso:** https://github.com/dani-saavedra/Ecommify_Database_Design

7.2 Contenido Disponible en el Repositorio

En este espacio se encuentran organizados los siguientes recursos técnicos:

- **Diccionario de Datos Completo:** Detalle exhaustivo de todas las tablas relacionales de PostgreSQL y colecciones documentales de MongoDB, especificando tipos de datos, restricciones de integridad, nulidades y propósitos de negocio.
- **Scripts DDL Preliminares (.sql):** Código fuente ejecutable para la creación del esquema relacional completo en PostgreSQL , incluyendo la activación de extensiones espaciales y de texto (PostGIS, pg_trgm) , y la lógica de particionamiento por rangos mensuales.

- **Scripts DML de Prueba (.sql):** Set de datos de ejemplo e inserciones coherentes para validación de flujos operacionales del marketplace.